

Health & Vitals: plan unificat v2 (revizuit critic)

Data: 5 octombrie 2026 **Bază:** audit complet, verificat direct în cod, pe 4 fronturi paralele: iOS/DailyCore/watchOS, Android, WinUI și C# partajat, Supabase și wearables. Fiecare afirmație de mai jos are sursă `fișier:linie`. **Status:** este doar un plan. Nu am modificat niciun fișier de cod.

IMPORTANT

Concluzia pe scurt: planul v1 avea direcția corectă (backend comun, cache local, paritate UI cu iOS), dar pornea de la premise greșite în puncte esențiale:

- datele **nu** ajung toate în Supabase azi;
- Android **nu** sincronizează nimic cu cloud-ul;
- iOS are bug-uri serioase în stratul de date, chiar dacă UI-ul e cel mai bun;
- există o problemă de securitate critică, care trebuie rezolvată înainte de orice altceva.

v2 corectează aceste premise și adaugă ce lipsea: contract de date, idempotență, fusuri orare, teste de paritate și criterii de acceptanță măsurabile.

1. Auditul planului v1

1.1 Ce rămâne valabil

- iOS este referința pentru **UI și experiență**: 5 sub-taburi, Sleep Studio, Stress Studio cu mascota, breathwork, Trends.
- Dualitatea `vitals` / `health_vitals` și trigger-ul greșit (sumă pe pași cumulativi, dată în UTC) sunt probleme reale.
- WinUI este mult în urmă și are nevoie de un hub unificat cu 5 taburi.
- Ideea generală de backend comun plus cache local.

1.2 Erori factuale în v1

Afirmația din v1	Realitatea verificată
„Toate sursele raportează în health_telemetry ”	Doar aplicația Apple Watch și ZeppOS scriu date raw. • iPhone nu urcă niciodată sample-uri HealthKit; scrie doar <code>vitals</code> derivate (<code>HealthDataService.swift:617</code>). • Android nu urcă nimic: <code>pushTelemetry / pushVitals</code> nu au niciun apelant (<code>HealthRemoteService.kt:12,42</code>). • WearOS și HarmonyOS nu trimit deloc date de sănătate.
Android „~85-90% paritate, Room cu <code>synced_at</code> funcțional”	UI-ul e la ~85%, dar stratul de date e la ~15-20% : • Citirile din Supabase eșuează silențios. <code>start_time</code> e declarat Long , iar Postgres trimite string ISO; în plus, filtrul trimite epoch-ms către o coloană <code>timestamp_tz</code> (<code>HealthModels.kt:141</code> , <code>HealthRemoteService.kt:32</code>). • Room nu e citit niciodată înapoi: se scrie cu <code>userId="local_health_connect"</code> și se citește cu <code>currentUserId</code> . • ID-urile sunt UUID random la fiecare citire, deci apar duplicate nelimitate. • Nu există WorkManager. • Migrarea Room e distructivă.
iOS „100% Gold Standard”	UI-ul e cel mai bun, dar datele au probleme: • Trends afișează valori fabricate (<code>generateHistoricalValue</code>) pentru zilele fără date, inclusiv la utilizatorii autentificați (<code>HealthDataService.swift:739,767</code>). • Upsert-ul în <code>vitals</code> eșuează după prima scriere a fiecărei zile: nu are <code>onConflict</code> , generează UUID nou și lovește <code>unique(user_id,date,type)</code> . • Nu există paginare: limita implicită PostgREST de 1000 de rânduri taie cele mai noi date. • Widget-urile sunt suprascrise cu ziua selectată, chiar dacă e din trecut. • „AI Coach” returnează texte predefinite după cuvinte-cheie, nu folosește Gemini.
„CREATE VIEW health vitals AS SELECT * FROM vitals pentru compatibilitate”	Ar strica trigger-ul existent , care face <code>INSERT ... ON CONFLICT</code> în <code>health_vitals</code> . Pe un view simplu nu se poate face <code>ON CONFLICT</code> .
„Supabase Realtime funcționează pe cele 3 tabele”	Canalele <code>realtime_health</code> și <code>realtime_telemetry</code> sunt respinse de server (<code>mac_logs2.txt:85</code> , „unmatched topic”). Nu există SQL care să adauge tabelele <code>health</code> în publicația <code>supabase_realtime</code> .
„Refactor SupabaseHealthService.cs ”	Fișierul e compilat și de aplicația MAUI legacy (iOS, Android, Mac, Windows; <code>Daily.csproj</code>). Orice schimbare o afectează și pe ea.

1.3 Ce lipsea complet din v1

1. Securitate (critic):

- Secretul de semnare JWT al proiectului e hard-codat în `supabase/migrations/20260903_watch_jwt_rpc.sql:19`, deci se află în istoricul git.
- `paired_watches` are politici RLS anon `SELECT/UPDATE USING(true)`: oricine are cheia publică poate citi token-urile de pairing ale tuturor utilizatorilor.

2. **Idempotență:** `health_telemetry` nu are nicio cheie unică. watchOS, Zepp și reinstalările produc duplicate.

3. **Normalizarea tipurilor:** există 5 variante pentru SpO2 (`spo2`, `blood_oxygen`, `oxygen_saturation`, `OxygenSaturation` ...) și 5 pentru HRV. MAUI scrie `Steps`, aplicațiile native scriu `steps`; cheia unică e case-sensitive, deci apar rânduri duplicate.

4. **Fus orar și DST:** ferestrele se calculează cu `+24h` și `-6h` fixe; Zepp ștampilează valorile punctuale cu ora telefonului; timezone-ul nu e stocat nicăieri.

5. **„Cine are datele?”:** un motor de calcul pe server nu poate funcționa, pentru că telefoanele (iOS și Android) nu urcă date raw.

6. **WinUI nu are senzori.** Are nevoie de un renderer, nu de motoare de calcul. v1 propunea portarea algoritmilor în C#.

7. **Testare de paritate:** lipseau fixture-urile comune și criteriile de acceptanță măsurabile.

8. Politici de store și GDPR:

- Apple 5.1.3: datele HealthKit pot sta pe server doar cu consimțământ și privacy policy, niciodată în iCloud.
- Google Play cere o declarație pentru datele Health Connect.
- GDPR: lipsesc politicile DELETE și `ON DELETE CASCADE`, deci utilizatorul nu își poate șterge datele.

9. **Constrângere de build:** WinUI nu se compilează pe macOS; verificarea are nevoie de o mașină Windows.

2. Starea reală pe fiecare platformă

2.1 iOS / DailyCore / watchOS

UI-ul este referința de păstrat 1:1.

- 14 view-uri principale, aproximativ 4.500 de linii.

- Ecrane: Overview, Sleep Studio, Stress Studio, Heart & Vitals, Trends.
- Interacțiuni:
 - navigare pe zile, DatePicker grafic, meniu de dispozitive;
 - swipe între sub-taburi, cu haptic; pull-to-refresh;
 - hypnogramă cu tooltip la tap;
 - breathwork cu 4 protocoale și mascota animată;
 - deep links `daily://health/sleep|stress`.

Probleme de date de reparat (UI-ul rămâne identic):

#	Problemă	Locație
1	Trends fabricate pentru zilele lipsă, la toți utilizatorii	HealthDataService.swift:739, 767–788
2	Upsert vitals fără onConflict și cu UUID-uri noi → eșuează tot batch-ul	HealthDataService.swift:596–621
3	Se urcă în vitals și înregistrări derivate/virtuale (stress-engine, RHR „computed”, Bubbles)	HealthDataService.swift:477–557
4	Pașii și durata somnului nu ajung niciodată în vitals	HealthDataService.swift:375
5	Fără paginare pe health_telemetry (zi și 7 zile)	HealthDataService.swift:251, 673
6	Widget-urile de somn și stres sunt rescrise cu ziua selectată, inclusiv din trecut	HealthDataService.swift:420, 560
7	Calorii active: câștigă ultimul segment primit de la ceas, nu suma zilei	HealthDataService.swift:380, 1076
8	Fără HKObserverQuery / background delivery; nici reload la revenirea în foreground	DailyApp.swift:24–32
9	Hidratarea vine din data selectată în Habits, nu din data din Health	HealthDataService.swift:492
10	Greutatea e ultimul sample din istoric, afișat ca valoare a zilei selectate	HealthKitManager.swift:216–245
11	isoDateFormatter fără en_US_POSIX ; ferestre greșite la schimbarea orei (DST)	HealthDataService.swift:73, 243
12	Duplicate între ceas și HealthKit („Apple Watch (Watch)” vs „Apple Watch”)	watchOS :181–193
13	Somnul agregat (fără stadii, de la ceas) are scorul plafonat la ~75	SleepClusteringEngine.swift (aggregate path)
14	Fallback care inventează ora de trezire 07:15 și o siestă la 14:00	SleepClusteringEngine.swift :119–190
15	Probleme de UX: • titlul „HEART RATE TODAY” apare și pe zile trecute; • tooltip-ul HR nu e conectat (cod mort); • DatePicker permite date viitoare; • isLoading nu e afișat nicăieri; • Resonance folosește 5 s, deși descrierea spune 5,5 s	—
16	SmartBriefing trimite stres 0 / „Calm” când nu există date	SmartBriefingService.swift:264

watchOS:

- face un `insert` simplu, fără deduplicare;
- la primul sync urcă **tot istoricul** (anchor `nil` , fără limită);
- două sync-uri pot rula concurent (background și scene-active), deci apar duplicate;
- somnul își pierde stadiile (`sleep` , în ore);
- tipurile nu sunt canonice: `hrv` , `spo2` .

2.2 Android

UI: ~85% din iOS. Componentele există: cele 5 taburi, Sleep Studio, Stress Studio, mascota, hypnograma, Trends.

Lipsesc față de iOS:

- cardul de preview pentru stres din Overview;
- swipe între taburi;
- iconițele per dispozitiv din meniu.

Date: aici e problema majoră.

#	Problemă	Locație
1	Citirile din Supabase nu funcționează: Long vs timestampz ISO, iar erorile sunt înghițite	HealthModels.kt:141–144, 218–220 ; HealthRemoteService.kt:32
2	Upload-ul nu există (cod mort); nici WorkManager	HealthRemoteService.kt:12, 42
3	Room nu e citit niciodată (userId diferit) și se umple de duplicate (UUID random)	HealthConnectManager.kt:126 ; HealthModels.kt:136
4	Health Connect fără paginare (pageToken); un singur try/catch pentru 8 tipuri, deci o permisiune refuzată oprește tot	HealthConnectManager.kt:115–317
5	Calorii și hidratare = ultimul interval, nu totalul zilei (fără Aggregate API)	HealthDataRepository.kt:387–410, 751
6	HRV: Health Connect oferă doar RMSSD, dar UI-ul și motorul de stres așteaptă SDNN (baseline 45 ms)	HealthConnectManager.kt:275 ; StressAnalysisEngine.kt:25
7	Dispozitivul e numele brut de pachet (com.sec.android.app.shealth). Rezultă: • ranking-ul de somn și isWearable greșite; • Huawei tratat ca dispozitiv cumulativ, deși trimite intervale	SleepClusteringEngine.kt:466
8	Date false: • Trends; • stres intraday mock; • widget-uri Glance cu valori hard-codate (88, 7h42m, HRV 45/48, „+34m”)	HealthDataRepository.kt:604 ; StressStudioView.kt:875 ; DailySleepGlanceWidget.kt:63–94
9	Încărcări suprapuse fără Mutex sau anulare; SimpleDateFormat partajat între corutine	HealthDataRepository.kt:172–358
10	Permițiuni lipsă: READ_HEALTH_DATA_HISTORY , READ_HEALTH_DATA_IN_BACKGROUND , intent de rationale (Android ≤ 13)	AndroidManifest.xml
11	Widget-urile citesc din memorie; la pornire „la rece” arată placeholder-e	Glance widgets
12	Doar 5 teste (stres). Niciun test pentru somn, pași sau mapper	core-health/src/test

2.3 WinUI (și C# partajat cu MAUI)

Ce există:

- două ferestre separate: HealthDetailPage (doar ziua de azi) și HealthTelemetryDetailPage (cu navigare pe zile);

- două widget-uri.

Ce lipsește:

- Stress Studio și motorul de stres;
- verdictul și sfaturile de somn;
- siestele (datele sunt încărcate, dar nu se afișează);
- selectorul de dispozitive, histograma pe 24 h, inelele, min/max/RHR;
- stările de încărcare, „fără date” și offline.

Algoritmii diferă de iOS:

- fereastra de somn e 20:00 → 14:00, iar pragul de gap e 90 min;
- nu există partiționare pe dispozitiv;
- scorul de somn e versiunea veche;
- **hypnograma e sintetizată** când lipsesc stadiile;
- pașii se calculează cu SUM, MAX și SUM, în 3 locuri diferite.

Alte probleme:

- Realtime nu funcționează („unmatched topic”), iar DELETE e no-op (ID-urile locale sunt MD5).
- Tabela `vitals` e rescrisă integral la fiecare pornire.
- `NullLogger` ascunde toate erorile.
- `GetHistoryAsync` are un bug de dată în UTC.
- SmartBriefing pe WinUI are **valori implicite false**: 2450 de pași, 7,5 h de somn, 68 bpm (`SmartBriefingService.cs:404-406`).

Puncte de sprijin pentru noul UI:

- Mica;
- dicționare de temă Dark/Light și pensule glass;
- Syncfusion Charts și Gauge;
- iconițe Tabler;
- controale desenate custom (`LungsControl`, `WaterLevelControl`).

`GlassWidgetContainer` **nu** e un material cu blur, ci un overlay de „murdărie” cu animație de ștergere. În plus, are un timer de 1 s care scrie `debug_glass.log`, adică un bug.

2.4 Supabase și wearables

Securitate:

- secretul JWT e în git;
- `paired_watches` are RLS deschis pentru anon; `watch_pairing_codes` este foarte probabil citibil anonim;
- RPC-urile `SECURITY DEFINER` nu verifică proprietarul și nu fixează `search_path` ;
- token-urile de ceas sunt valabile 10 ani și nu pot fi revocate;
- **Postgres 17 nu mai are `pgjwt`** , de care depinde `generate_watch_token` . Trebuie verificat live: s-ar putea ca pairing-ul să cadă deja pe un token care expiră într-o oră.

Schemă:

- `health_telemetry` : fără cheie unică și fără politici UPDATE/DELETE;
- `vitals` : fără DELETE;
- fără `ON DELETE CASCADE` și fără politică de retenție;
- `supabase/migrations/` nu conține nicio tabelă health. Tot DDL-ul e în `Docs/` , aplicat manual, iar baza live deja diferă de el.

Trigger-ul de agregare:

- calculează data în UTC;
- adună pașii cumulativi trimiși de Zepp;
- last-write-wins după ordinea de inserare;
- ignoră SpO2, somnul, stresul și RHR;
- scrie în `health_vitals` , pe care aplicațiile native nu o citesc.

ZeppOS:

- valorile punctuale (HR, SpO2, stres) sunt ștampilate cu ora sync-ului;
- cache-ul de deduplicare folosește ziua UTC;
- `source_device` e hard-codat „Zepp OS Watch”.

3. Decizia de arhitectură: unde trăiește logica?

3.1 Opțiunile analizate

Opțiune	Implementări de întreținut	Pro	Contra
A. Motoare native pe fiecare platformă (inclusiv un port C# pentru WinUI)	Swift + Kotlin + C#	Funcționează complet offline	• 3 implementări care pot diverge (deja se întâmplă) • WinUI ar descărca mii de rânduri raw pe zi
B. Bibliotecă partajată (Rust + UniFFI → Swift, Kotlin, C#; sau KMP)	1 (Rust)	O singură sursă de adevăr	• Rescriere totală, toolchain complex • KMP nu ajunge în .NET; binding-ul Rust → C# e third-party • Risc mare pentru aplicația iOS, care azi funcționează
C. Motor canonic pe server (Edge Function) + clienți subțiri	TS (canonic) + Swift și Kotlin existente, ca fallback offline	• Aceleași numere peste tot, persistate • WinUI, widget-urile, Smart Briefing și ceasurile citesc același rezumat • WinUI nu mai are nevoie de niciun motor	• Telefoanele trebuie să urce date raw • Latență pentru ziua curentă • Încă un limbaj (TS)

3.2 Recomandare: varianta C (hibrid)

Principiul: **datele raw în cloud, rezumatul canonic calculat pe server, motoarele locale verificate cu aceleași teste.**

De ce e varianta cu cea mai mare șansă de reușită:

1. **WinUI devine renderer pur.** Nu se poartază niciun algoritm în C# (desktopul nu are senzori), deci dispare cea mai mare sursă de divergență.
2. **Numerele sunt persistate o singură dată**, în `health_daily_summary`. Telefonul, desktopul, widget-urile, Smart Briefing și ceasurile afișează exact aceleași valori.
3. **iOS și Android își păstrează motoarele existente**, dar doar pentru offline și pentru afișarea instantanee a zilei curente (valori provizorii). Paritatea cu serverul e garantată de **golden**

fixtures comune, rulate în CI pe Swift, Kotlin și TS.

4. **Contractul e stabil.** Clienții primesc un singur JSON per zi, cu tot ce afișează UI-ul iOS:

- sesiunile și stadiile de somn, hypnograma, siesta, verdictul și sfaturile;
- pașii pe ore;
- pulsul redus la valori pe minut, plus zonele;
- stresul pe ore și starea mascotei;
- vitalele, cu sursa și vechimea fiecăreia.

NOTE

Alternativa mai ieftină (dacă vrei să eviți Edge Function-ul): WinUI calculează local, cu un port C# al motorului, testat cu aceleași golden fixtures. Costul de implementare e similar, dar riscul de divergență e mai mare și rezumatul nu e persistat central. O recomand doar dacă motorul pe server e exclus din alte motive (cost, latență).

4. Contractul de date (fundatia; fără el nimic nu va fi „perfect”)

4.1 Registru canonic de metrici (`HealthSpec/metric_registry.json`)

- **Un singur fișier** în repo. Din el se **generează** enum-urile pentru Swift, Kotlin, C# și TS, deci nu mai există mapări scrise de mână.
- Pentru fiecare metrică se definesc:
 - **cheia canonică** (de ex. `oxygen_saturation`);
 - **aliasurile** acceptate la ingestie (`spo2` , `blood_oxygen` , `OxygenSaturation` ...);
 - **unitatea canonică** și conversiile (somnul: ore, minute sau secunde → **secunde**);
 - **semantica**: `interval_delta` , `cumulative_daily` , `spot` sau `session_stage` ;
 - **regula de agregare zilnică**: `sum` , `max_of_cumulative` , `latest` , `mean` sau `min` ;
 - **fereastra de atribuire**: ziua calendaristică locală sau fereastra nocturnă D-1 18:00 → D 18:00.
- **HRV**: `hrv_sdn` și `hrv_rmssd` sunt **metrici separate**, fiecare cu baseline-ul său în motorul de stres. RMSSD nu se mai compară cu un baseline de SDNN.

4.2 Identitatea sursei (`health_sources`)

- `source_id` stabil per utilizator: hash din platformă + bundle/pachet + model de dispozitiv.

- Câmpuri:
 - `device_kind`: `wearable_watch`, `ring`, `phone`, `scale`, `manual`, `merged_store` sau `virtual`;
 - `display_name`: „Amazfit Balance”, „Oura Ring”, „Pixel Watch 3”...;
 - `vendor`, `priority_sleep`, `priority_activity`.
- Dispare potrivirea fragilă pe subșiruri (`"balance"`, `"health"`, `"watch"`).
- Pe Android, maparea pachet → dispozitiv devine o tabelă explicită:
 - `com.sec.android.app.shealth` → „Samsung Health”;
 - `com.ouraring.oura` → „Oura Ring”;
 - `com.huami.watch.hmwatchmanager` → „Zepp”;
 - etc.

4.3 Granularitatea la ingestie (controlează volumul)

Metrică	Ce se urcă	Volum estimat pe zi
Pași, calorii active	Bucket-uri orare per sursă , cu upsert (ora curentă se actualizează pe măsură ce crește)	$\leq 24 \times \text{nr. surse}$
Puls	Sample-uri raw (în antrenament, medie pe minut)	150–1500
Somn	Intervalele de stadiu raw, cu stadiul păstrat	30–80
Vitale punctuale (RHR, HRV, SpO2, respirație, greutate, temperatură)	Sample raw cu ora reală a măsurării	< 50
Stres și PAI de la Zepp	Sample cu ora reală a măsurării (nu ora sync-ului)	< 50

4.4 Idempotență

- Fiecare rând primește un `external_id` :
 - UUID-ul `HKObject` (trebuie verificat că se păstrează între ceas și iPhone);
 - `metadata.id` din Health Connect;
 - pentru bucket-uri, un hash determinist din sursă + tip + start.
- Index unic (`user_id`, `source_id`, `type`, `external_id`) . Clienții noi folosesc `upsert(onConflict: ...)` .

- **Compatibilitate cu ceasurile deja instalate**, care fac `insert` simplu: un trigger `BEFORE INSERT` returnează `NULL` pentru rândurile duplicate, **în loc** să arunce eroare de unicitate. Altfel, un singur duplicat ar bloca tot batch-ul, iar ceasul ar reîncerca la nesfârșit fără să avanseze anchor-ul.

4.5 Timp și fus orar

- Toate timestamp-urile se trimit ca **ISO-8601 UTC** și se stochează ca `timestampz`.
- Fiecare rând are `tz_offset_min`, iar profilul are `timezone` (IANA).
- `local_date` se calculează **pe server**, din fusul orar al rândului, deci rămâne corect și la călătorii.
- Ferestrele se construiesc cu aritmetică de calendar (`date(byAdding: .day) / ZonedDateTime`), nu cu `+24h`.
- Golden fixtures includ zilele cu schimbare de oră: o zi de 23 h și una de 25 h.

5. Schema Supabase v2 (propunere)

WARNING

Înainte de orice migrare am nevoie de un **dump al schemei live**: politici, trigger-e, funcții, publicații, nulabilitatea coloanelor. Repo-ul nu reflectă baza live. Variante:

- îmi dai parola DB sau acces prin Supabase CLI; sau
- rulezi tu câteva interogări read-only pe care ți le pregătesc.

Obiect	Acțiune
health_telemetry	Îl evoluăm, nu îl înlocuim, pentru că ceasurile deja instalate scriu aici: - coloane noi: external_id, source_id, semantics, tz_offset_min, local_date, ingested_at; - trigger BEFORE INSERT care normalizează (alias → canonic, unitate, sursă) și deduplicatează silențios; - index (user_id, type, start_time); - RLS: UPDATE și DELETE pentru proprietar; - ON DELETE CASCADE.
health_sources	Tabelă nouă (vezi §4.2).
health_day_dirty	Tabelă nouă (user_id, local_date, dirty_since), completată de un trigger AFTER INSERT. Înlocuiește trigger-ul de agregare actual.
health_daily_summary	Tabelă nouă, cu (user_id, local_date) unic: - engine_version, computed_at, raw_watermark, summary_jsonb; - coloane scalare pentru Trends: steps, active_kcal, sleep_asleep_s, sleep_score, stress_avg, rhr, hrv_sdn, hrv_rmssd, weight, spo2; - RLS pe proprietar; Realtime activat.
vitals	Rămâne pentru compatibilitate (MAUI). Aplicațiile native nu mai scriu aici date derivate. Adăugăm politica DELETE și cascade, și normalizăm rândurile scrise în PascalCase.
health_vitals + trigger-ul vechi	Trigger-ul se dezactivează după ce health_daily_summary funcționează. Tabela rămâne temporar read-only (o citește doar MAUI), apoi se șterge.
Retenție	Pulsul raw mai vechi de N zile → medii pe minut. Somnul și vitalele se păstrează integral. Decizia ta: N = 400 de zile?
supabase/migrations/	Devine sursa de adevăr: tot DDL-ul health (actual și v2), versionat.

API pentru clienți:

- GET /functions/v1/health-day?date=YYYY-MM-DD : recalculează ziua dacă e marcată dirty, apoi returnează rezumatul.
- GET /functions/v1/health-range?from=&to= : returnează doar coloanele scalare, pentru Trends.
- pg_cron, noaptea: finalizează zilele încheiate.

6. Workstream-uri pe platforme

WS-0 · Securitate (blocant; nu ține de Health, dar trebuie făcut primul)

1. Rotim secretul JWT (sau migrăm pe JWT signing keys) și îl mutăm din fișier în Vault / `app.settings`.
 - ⚠️ Asta invalidează toate sesiunile și impune re-pairing pentru ceasuri.
2. Înlocuim politicile anon de pe `paired_watches`, `watch_pairings` și `watch_pairing_codes` cu RPC-uri `SECURITY_DEFINER` care verifică proprietarul și au `search_path` fix. Claim-ul trece prin `claim_orbit_pin`, care azi nu e folosit.
3. Verificăm `extensions.sign` pe PG17. Token-urile pentru ceas primesc durată limitată și refresh real.
4. Scoatem `supabase/.temp/` din git.

WS-1 · Contract și schemă

- Registrul de metrice și de surse, plus generatorul de cod.
- Dump-ul schemei live, migrațiile v2 și testele RLS.

WS-2 · Ingestie: toate sursele ajung în cloud

iOS:

- `HKAnchoredObjectQuery` + `HKObserverQuery` + `enableBackgroundDelivery` + `BGAppRefreshTask`;
- upsert idempotent, cu bucket-uri orare pentru pași și calorii;
- ecran de consimțământ și actualizarea Privacy Nutrition Label.

Android:

- `TimestampSerializer` pe modelele health (există deja, în `HabitModels.kt:85`);
- ID-uri din `metadata.id`;
- paginare cu `pageToken` și try/catch separat pentru fiecare tip;
- `AggregateRequest` pentru pași, calorii și hidratare;
- `CoroutineWorker` periodic, cu constrângere de rețea;
- permisiunile `READ_HEALTH_DATA_IN_BACKGROUND` și `HISTORY`, plus intent-ul de rationale;
- migrații Room reale, nu distructive.

watchOS:

- `external_id` = UUID-ul HK; tipuri canonice;
- **stadiile de somn păstrate** (inclusiv awake);
- lock împotriva sync-urilor concurente;
- primul sync limitat la 30 de zile; batch-uri de maximum 500 de rânduri.

ZeppOS:

- ora reală a măsurării;
- `semantics=cumulative_daily` pentru pași și calorii;
- cache de deduplicare pe ziua **locală**;
- `source_device` = modelul real;
- verificat `bo.getCurrent()` pentru SpO2.

WearOS / HarmonyOS: în afara scopului v1. Pe WearOS, datele ajung oricum prin Health Connect de pe telefon. **Confirmi?**

WS-3 · Motorul canonic (Spec v1 + Edge Function + golden fixtures)

- Scriem `HealthSpec/ENGINE_SPEC_v1.md`, extras din codul Swift. Spec-ul rezolvă explicit divergențele și bug-urile, cu decizia ta pentru fiecare (vezi §9).
- Portăm motorul în TypeScript (Deno), ca `health-engine`.
- **Golden fixtures:** 15–25 de zile reale anonimizate, fiecare cu input (raw) și output așteptat (JSON). Cazuri acoperite:
 - o noapte Apple Watch + Oura;
 - o zi Zepp cu pași cumulativi;
 - o zi Samsung Health prin Health Connect;
 - o noapte cu siestă și o noapte fragmentată;
 - o zi fără date;
 - DST primăvara și toamna;
 - o zi cu călătorie între fusuri orare;
 - multi-device, cu ceasul pe încărcător.
- Același set rulează în **Swift (`swift test`), Kotlin (JUnit) și Deno test**. Orice divergență blochează merge-ul.

WS-4 · iOS: stratul de date (UI-ul nu se schimbă)

- Eliminăm `generateHistoricalValue` pentru utilizatorii reali. Zilele fără date apar ca bare goale sau „—”.

- Oprim upsert-ul în `vitals` . Aplicația citește `health_daily_summary` și îl păstrează în cache local (un JSON per zi în App Group).
- Widget-urile se actualizează **doar pentru ziua de azi**. Facem reload la revenirea în foreground și rollover la miezul nopții.
- Hidratarea se ia pe data din Health. Greutatea apare cu eticheta de vechime („acum 12 zile”).
- Reparații UX: titlul HR pe zilele trecute, DatePicker limitat cu `in: ...today` , indicator de încărcare.
- **Decizie necesară (§9):** AI Coach real cu Gemini sau redenumit „Ghid”.

WS-5 · Android: date + golurile de UI

- Același model ca pe iOS: rezumat de pe server, Room ca cache (`health_daily_summary_entity`), motorul local folosit doar provizoriu.
- Eliminăm toate datele false: Trends, stres intraday mock, widget-urile Glance.
- Widget-uri cu stare persistentă (Glance `PreferencesGlanceStateDefinition` sau `DataStore`).
- Concurență: `Mutex` sau `Job` anulabil, plus verificarea datei selectate.
- UI de completat:
 - `StressOverviewPreviewCard` ;
 - swipe între taburi, cu haptic;
 - iconițe pe dispozitive;
 - banner Health Connect pentru „provider update required”.
- Verificare: întâi pe emulator, apoi pe Pixel 9 Pro, Z Fold 8 „RADAR” și S25 Edge.

WS-6 · WinUI: un Health Hub nou (renderer pur)

- **Serviciu nou, doar în proiectul WinUI:** `HealthHubService` . Citește rezumatele, păstrează un cache SQLite (`health_daily_summary`) și folosește Realtime cu topic corect.
- **Nu modificăm `SupabaseHealthService.cs`** , ca aplicația MAUI legacy să rămână neatinsă.
- **`HealthHubPage`** : o singură pagină în `DetailWindow` .
 - Header: < dată >, calendar, Today, meniu de dispozitive.
 - 5 taburi:
 1. **Overview:** inele de activitate (control desenat), histogramă pe 24 h, preview pentru somn și stres, grilă de 8 tile-uri.
 2. **Sleep Studio:** inel de scor animat, `SleepVerdictCard` , sfaturi, hypnogramă (portată din codul Canvas existent), grilă de stadii, siestă.

3. Stress Studio:

- **mascata portată în XAML / Win2D**, cu aceleași forme și animații;
- bara autonomă, cele 4 drivere, ritmul orar;
- **player de breathwork** cu aceleași 4 protocoale și aceleași durate.

4. **Heart & Vitals:** curbă intraday cu tooltip, min/max/RHR, 4 zone, grilă de vitale cu sursă și vechime.

5. **Trends:** 7 carduri verticale cu capsule pe 7 zile și AVG/HIGH/LOW/TOTAL.

- **Design:** set de design tokens comun, extras din iOS (`ThemeColors.swift` : culori, raze, spațieri, fonturi).
 - `GlassCard` pe WinUI = Acrylic pe Mica + bordură luminoasă.
 - Curățăm timer-ul de debug din `GlassWidgetContainer` .
- **Widget-urile din dashboard** se unifică într-un singur `HealthWidget` , care citește rezumatul zilei de azi.
- **SmartBriefing pe WinUI:** eliminăm valorile implicite false.
- **Verificare:** necesită o mașină Windows (vezi §9).

WS-7 • Validare și documentare

- Matrice de dispozitive și paritate pe screenshot-uri (§7).
- Documentare în `Docs/Features/` .
- Corectăm documentele care conțin afirmații false (lista e în audit).

7. Testare și criterii de acceptanță

Criteriu	Țintă măsurabilă
Aceeși zi pe iOS, Android și WinUI	Pași, scor de somn, durată de somn, stres mediu, RHR și HRV identice (toleranță 0; ±1 doar la rotunjiri documentate)
Paritatea motoarelor	100% din golden fixtures trec pe Swift, Kotlin și TS
Date false	Zero. Verificat automat în CI: • grep după <code>generateHistoricalValue</code>, <code>generateMockIntraday</code> și valori hard-codate în widget-uri; • teste unitare pentru stările „fără date”
Latență de sincronizare	Un sample nou pe telefon apare pe WinUI în < 2 min (aplicația în foreground)
Deschidere offline	Ultimele 30 de zile se încarcă din cache în < 300 ms , fără rețea
Idempotență	3 re-sync-uri complete → același număr de rânduri
Paritate vizuală	Checklist per tab (componente, ordine, culori, stări „fără date” și de încărcare) + screenshot-uri iOS ↔ Android ↔ WinUI, revizuite de tine
DST și călătorie	Fixture-urile dedicate trec
Securitate	Teste RLS: utilizatorul A nu vede datele lui B; anon nu vede nimic

8. Roadmap realist

Total: ~55–80 de zile de lucru efectiv. Dacă P5 și P6 rulează în paralel, durata calendaristică scade la ~8–11 săptămâni.

Fiecare etapă livrează ceva utilizabil:

- după P0, sistemul e sigur;
- după P2, toate datele ajung în cloud;
- după P4, iOS e corect și fără date false;
- după P5, Android e corect;
- după P6, WinUI ajunge la paritate.

9. Decizii deschise (am nevoie de răspunsul tău)

1. **Arhitectura:** motor canonic pe server (recomandat) sau port C# local pe WinUI?
2. **iOS:** ai spus „nu vreau să schimb nimic”. UI-ul rămâne identic, dar **stratul de date trebuie reparat** (trends false, upsert care eșuează, upload raw). Ești de acord?
3. **Fallback-urile din Spec v1** (comportamentul iOS actual vs. varianta „onestă”):
 - Ora de trezire inventată (07:15) și siesta de la 14:00, când avem doar durata → afișăm **doar durata, fără ore**?
 - Somnul agregat fără stadii (de la ceas), plafonat la ~75 → scor marcat „parțial” sau scor ponderat doar pe durată și eficiență?
 - RHR = min + 4 când lipsește → afișăm „—”?
 - Calorii = pași × 0,042 → „—” sau eticheta „estimat”?
 - Zonele HR: după numărul de sample-uri sau **după timp** (mai corect)?
4. **AI Sleep Coach:** integrare reală cu Gemini (cu costuri și consimțământ) sau îl redenumim „Ghid de somn”?
5. **Securitate:** pot planifica rotirea secretului JWT? Implică re-pairing pentru toate ceasurile.
6. **Acces la Supabase live:** parolă DB / Supabase CLI, sau rulezi tu interogările read-only pregătite de mine?
7. **Windows:** ai o mașină sau un VM Windows pentru build și testare WinUI? Regulile din AGENTS.md acoperă iOS și Android, dar nu WinUI.
8. **Retenția pulsului raw:** după câte zile îl reducem la medii pe minut (propun 400)?
9. **Wearables în v1:** WearOS și HarmonyOS rămân în afara scopului (datele vin prin Health Connect)?
10. **MAUI legacy:** mai e folosit? Dacă nu, îl înghețăm sau îl scoatem. Asta simplifică WinUI și elimină scrierile în PascalCase.

10. Registru de riscuri

Risc	Impact	Mitigare
UUID-ul HKObject nu se păstrează între ceas și iPhone	Duplicate ceas ↔ telefon	Motorul alege o singură sursă per metrică (după prioritate) și nu adună; testat pe dispozitive reale în P2
Schema live diferă de cea din repo	Migrări care eșuează	Dump obligatoriu în P1; migrări testate pe un branch Supabase
Rotirea JWT rupe ceasurile	Sync-ul se oprește temporar	Fereastră planificată; fluxul de re-pairing testat dinainte
Latența Edge Function pentru ziua curentă	UI lent	Afișăm întâi valoarea provizorie din motorul local, apoi o înlocuim cu cea canonică. Fixture-urile garantează că valorile coincid
Pe Samsung, Health Connect nu expune HRV/RHR	Tile-uri goale	Stare explicită „indisponibil pe acest dispozitiv”, documentată
Review App Store / Play pentru date de sănătate pe server	Aplicație respinsă	Consimțământ explicit, privacy policy și formularul Data Safety actualizate în P2
WinUI nu se poate testa pe Mac	Regresii nedetectate	Mașină Windows; logica de date într-o bibliotecă <code>net10.0</code> , testabilă pe Mac (xUnit)
Volumul de puls raw	Cost și performanță	Bucket-uri, retenție, index <code>(user_id, type, start_time)</code>

TIP

Șansa de reușită:

- **Peste 85–90%** ca rezultatul final să arate și să se comporte identic pe toate platformele, dacă P0–P1 sunt făcute riguros: schemă live verificată, contract generat, fixtures comune.
- **Sub 50%** fără contract și fixtures (abordarea v1), pentru că divergențele de azi s-ar reproduce.